Task 1

```java
class Solution {
    public int[] sortedSquares(int[] nums) {
        int n = nums.length;
        int[] result = new int[n];
        int left = 0;
        int right = n - 1;

        // Fill the result array from back to front
        for (int i = n - 1; i >= 0; i--) {
            if (Math.abs(nums[left]) > Math.abs(nums[right])) {
                result[i] = nums[left] * nums[left];
                left++;
            } else {
                result[i] = nums[right] * nums[right];
                right--;
            }
        }

        return result;
    }
}
```

Output:

```
Input
nums =
[-4,-1,0,3,10]
Output
[0,1,9,16,100]
```

TASK 2

```java
import java.util.HashMap;

public class Solution {
    public boolean containsNearbyDuplicate(int[] nums, int k) {
        HashMap<Integer, Integer> seen = new HashMap<>();

        for (int i = 0; i < nums.length; i++) {
            // Check if the number was seen before and is within the distance k
            if (seen.containsKey(nums[i]) && i - seen.get(nums[i]) <= k) {
                return true;
            }
            // Update the map with the latest index of the number
            seen.put(nums[i], i);
        }

        return false;
    }
}
```

OUTPUT:

```
Input
nums =
[1,2,3,1]
k =
3
Output
true
```

TASK 3

```java
public class Solution {
    public int[] runningSum(int[] nums) {
        // Start from index 1 and add the previous element's value to the current
element
        for (int i = 1; i < nums.length; i++) {
            nums[i] += nums[i - 1];
        }
        return nums;
    }
}
```

OUTPUT:

```
Input
nums =
[1,2,3,4]
Output
[1,3,6,10]
```

TASK 4

```
public class Solution {
    public int[] getSumAbsoluteDifferences(int[] nums) {
        int n = nums.length;
        int[] result = new int[n];

        // Calculate the total sum of the array first
        int totalSum = 0;
        for (int num : nums) {
            totalSum += num;
        }

        int leftSum = 0;
        for (int i = 0; i < n; i++) {
            // Right sum is total sum minus left sum and the current element
            int rightSum = totalSum - leftSum - nums[i];

            // Elements to the left
            int leftCount = i;
            // Elements to the right
            int rightCount = n - 1 - i;

            // Calculate absolute differences dynamically
            int leftTotal = (nums[i] * leftCount) - leftSum;
            int rightTotal = rightSum - (nums[i] * rightCount);

            result[i] = leftTotal + rightTotal;

            // Update left sum for the next iteration
            leftSum += nums[i];
        }
```

```
        return result;
    }
}
```

OUTPUT:

```
Input
nums =
[2,3,5]
Output
[4,3,5]
```

```
public class Solution {
    public int search(int[] nums, int target) {
        int left = 0;
        int right = nums.length - 1;

        while (left <= right) {
            int mid = left + (right - left) / 2;
            if (nums[mid] == target) {
                return mid;
            } else if (nums[mid] < target) {
                left = mid + 1;
            } else {
                right = mid - 1;
            }
        }
        return -1;
    }
}
```

OUTPUT:

Input

nums =
[-1,0,3,5,9,12]
target =
9
Output
4

```
public class Solution {
    // 1. The method MUST be directly inside the Solution class
    public void sortColors(int[] nums) {
        int low = 0;
        int mid = 0;
        int high = nums.length - 1;

        while (mid <= high) {
            if (nums[mid] == 0) {
                swap(nums, low, mid);
                low++;
                mid++;
            } else if (nums[mid] == 1) {
                mid++;
            } else if (nums[mid] == 2) {
                swap(nums, mid, high);
                high--;
            }
        }
    } // Closes sortColors

    // 2. The helper method must also be inside the Solution class
    private void swap(int[] nums, int i, int j) {
        int temp = nums[i];
        nums[i] = nums[j];
        nums[j] = temp;
    } // Closes swap

}
```

OUTPUT:

```
Input
```

nums =
[2,0,2,1,1,0]
Output
[0,0,1,1,2,2]

```
public class Solution {
    public int maxProfit(int[] prices) {
        // Base case: if there's less than 2 days, no profit can be made
        if (prices == null || prices.length < 2) {
            return 0;
        }

        int minPrice = prices[0];
        int maxProfit = 0;

        for (int i = 1; i < prices.length; i++) {
            // Update the minimum price if a cheaper buying day is found
            if (prices[i] < minPrice) {
                minPrice = prices[i];
            } else {
                // Calculate potential profit and update max profit if it's higher
                int potentialProfit = prices[i] - minPrice;
                if (potentialProfit > maxProfit) {
                    maxProfit = potentialProfit;
                }
            }
        }

        return maxProfit;
    }
}
```

OUTPUT:

```
Input
prices =
[7,1,5,3,6,4]
Output
5
```

```
public class Solution {
    public int firstUniqChar(String s) {
        int[] freq = new int[26];

        // Step 1: Count the frequency of each character
        for (int i = 0; i < s.length(); i++) {
            freq[s.charAt(i) - 'a']++;
        }

        // Step 2: Find the first character with a frequency of 1
        for (int i = 0; i < s.length(); i++) {
            if (freq[s.charAt(i) - 'a'] == 1) {
                return i;
            }
        }

        return -1;
    }
}
```

OUTPUT:

```
Input
s =
"leetcode"
Output
0
```

```
public class Solution {
    public void moveZeroes(int[] nums) {
        int insertPos = 0;

        // Step 1: Shift all non-zero elements to the front of the array
        for (int i = 0; i < nums.length; i++) {
            if (nums[i] != 0) {
                nums[insertPos] = nums[i];
                insertPos++;
            }
        }

        // Step 2: Fill the remaining indices with zeroes
        while (insertPos < nums.length) {
            nums[insertPos] = 0;
            insertPos++;
        }
    }
}
```

OUTPUT:

```
Input
nums =
[0,1,0,3,12]
Output
[1,3,12,0,0]
```

```
public class Solution {
    public void reverseString(char[] s) {
        int left = 0;
        int right = s.length - 1;

        while (left < right) {
            // Swap characters at left and right pointers
            char temp = s[left];
            s[left] = s[right];
            s[right] = temp;

            // Move pointers toward the middle
            left++;
            right--;
        }
    }
}
```

OUTPUT:

```
Input
s =
["h","e","l","l","o"]
Output
["o","l","l","e","h"]
```